LAPORAN PRAKTIKUM 5 SISTEM INFORMASI GEOGRAFIS
INDEXING SPASIAL DAN OPTIMASI QUERY

Disusun Oleh:
Anselmus Herpin Hasugian (123140020)

Dosen Pengampu:

Muhammad Habib Algifari, S.Kom., M.T.I.

Alya Khairunnisa Rizkita, S.Kom., M.Kom.

PROGRAM STUDI TEKNIK INFORMATIKA

INSTITUT TEKNOLOGI SUMATERA

2026

BAB 1.

PENDAHULUAN

1.1. Latar Belakang

Seiring bertambahnya volume data spasial, eksekusi query pencarian geometri menjadi semakin berat dan lambat jika mesin basis data harus memindai setiap baris tabel satu per satu (Sequential Scan). Untuk mengatasi masalah skalabilitas ini, PostGIS menyediakan arsitektur GiST (Generalized Search Tree) yang menggunakan konsep Bounding Box (kotak pembatas) dan R-Tree untuk mengindeks data geometri. Praktikum ini bertujuan untuk melakukan benchmarking secara empiris guna mengukur efisiensi komputasi sebelum dan sesudah penerapan Spatial Index, serta mempraktikkan teknik optimasi kode SQL spasial dari anti-pattern menjadi best practice.

1.2. Tujuan

1. Memahami Urgensi Indexing: Menganalisis bagaimana volume data spasial memengaruhi performa query dan mengapa indexing menjadi solusi krusial dalam manajemen basis data GIS.
2. Implementasi GiST Index: Mempraktikkan pembuatan dan pengelolaan Spatial Index menggunakan metode GiST (Generalized Search Tree) pada berbagai tipe geometri (Point, LineString, dan Polygon).
3. Analisis Query Plan: Menggunakan perintah EXPLAIN ANALYZE untuk membedah rencana eksekusi (query plan) dan membandingkan metode Sequential Scan dengan Index Scan.
4. Optimasi Query Spasial: Mengidentifikasi anti-pattern pada penulisan SQL spasial dan melakukan refactoring menggunakan fungsi yang mendukung bounding box (seperti ST_DWithin).
5. Pengukuran Efisiensi (Benchmarking): Mendokumentasikan peningkatan performa (speedup) secara kuantitatif berdasarkan perbandingan waktu eksekusi sebelum dan sesudah optimasi.

BAB 2.

BENCHMARK AWAL (TANPA INDEX)

Pada tahap ini, sistem mengandalkan metode *Sequential Scan*. Fungsi EXPLAIN ANALYZE digunakan untuk merekam waktu murni (*Execution Time*) yang dibutuhkan mesin dalam menyelesaikan komputasi spasial.

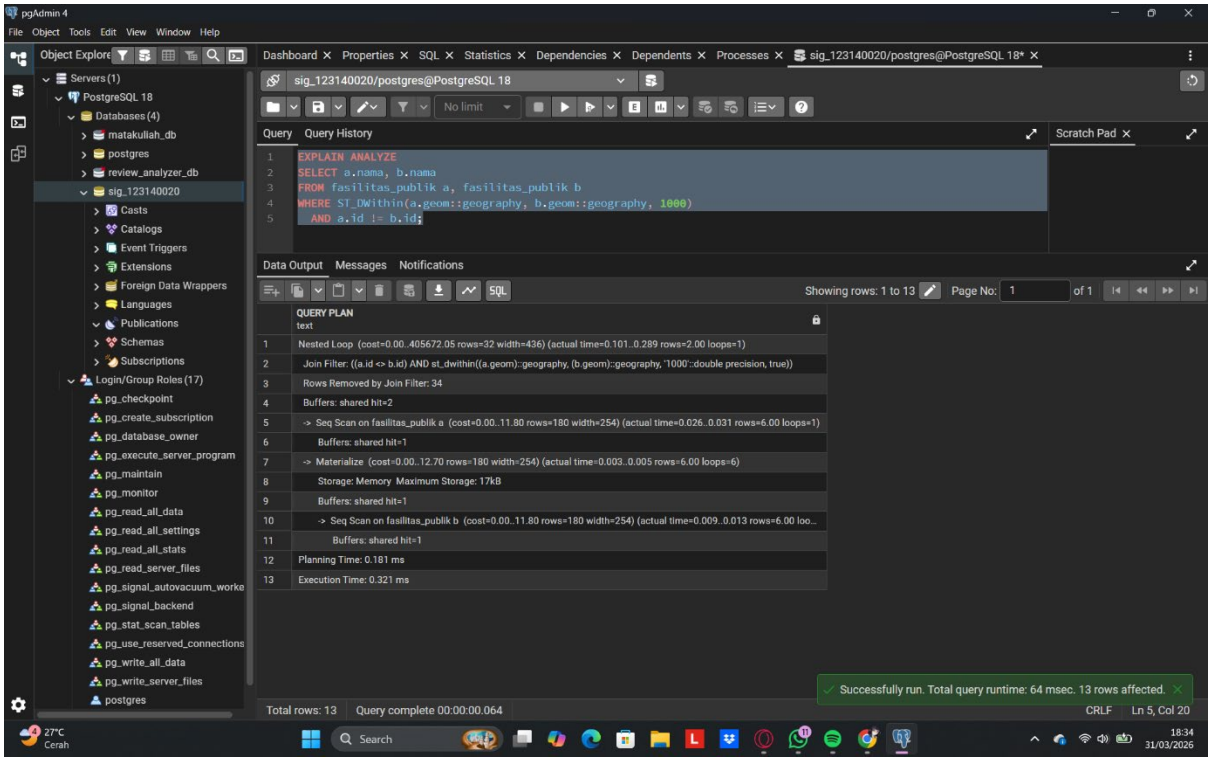
2.1. Query Jarak (Fasilitas Publik)

Mencari fasilitas dalam radius 1000 meter.

Execution Time: **0.321 ms**

Query:

```
EXPLAIN ANALYZE
SELECT a.nama, b.nama
FROM fasilitas_publik a, fasilitas_publik b
WHERE ST_DWithin(a.geom::geography, b.geom::geography, 1000)
  AND a.id != b.id;
```



Gambar 2.1 Tampilan Querry Jarak Fasilitas Publik

Hasil eksekusi mencatat waktu sebesar **0.321 ms**. Waktu ini relatif cepat karena geometri titik (*Point*) memiliki struktur matematika yang sederhana, sehingga komputasi murni tanpa indeks belum terlalu membebani *server*.

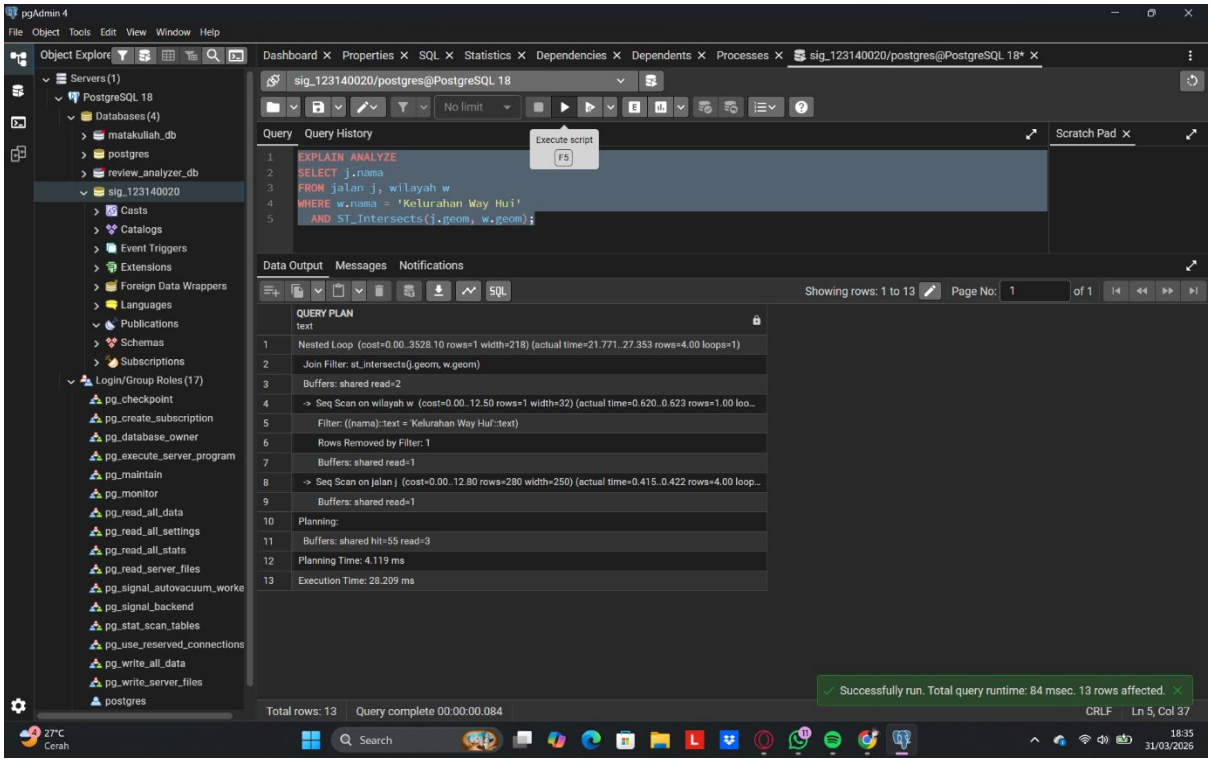
2.2. Query Persinggungan (Jalan vs Wilayah)

Mencari jalan yang memotong poligon kelurahan.

Execution Time: **28.209 ms**

Query:

```
EXPLAIN ANALYZE
SELECT j.nama
FROM jalan j, wilayah w
WHERE w.nama = 'Kelurahan Way Hui'
  AND ST_Intersects(j.geom, w.geom);
```



Gambar 2.2. Tampilan Query Persinggungan Jalan dan Wilayah

Komputasi melonjak tajam menjadi **28.209 ms**. Hal ini disebabkan mesin harus mengevaluasi setiap titik belokan (vertex) pada garis jalan (*LineString*) untuk melihat apakah bersinggungan dengan batas poligon wilayah secara *brute-force*.

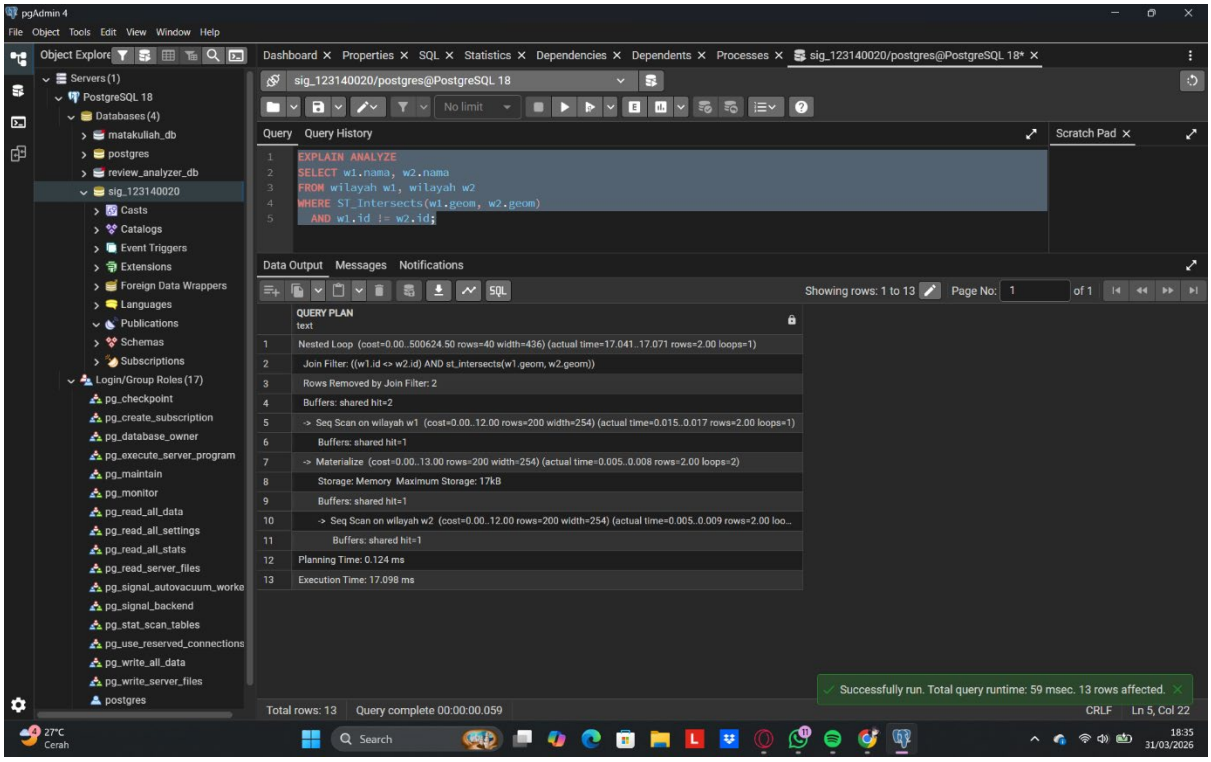
2.3. Query Tumpang Tindih (Wilayah vs Wilayah)

Mengevaluasi irisan antar poligon wilayah.

Execution Time: **17.098 ms**

Query:

```
EXPLAIN ANALYZE
SELECT w1.nama, w2.nama
FROM wilayah w1, wilayah w2
WHERE ST_Intersects(w1.geom, w2.geom)
 AND w1.id != w2.id;
```



Gambar2.3. Tampilan Query Tumpang Tindih Wilayah dan Wilayah

Waktu eksekusi tercatat sebesar **17.098 ms**. Mengevaluasi irisan luasan antar poligon memberikan beban menengah yang cukup signifikan pada sistem sebelum adanya struktur *R-Tree*.

BAB 3.

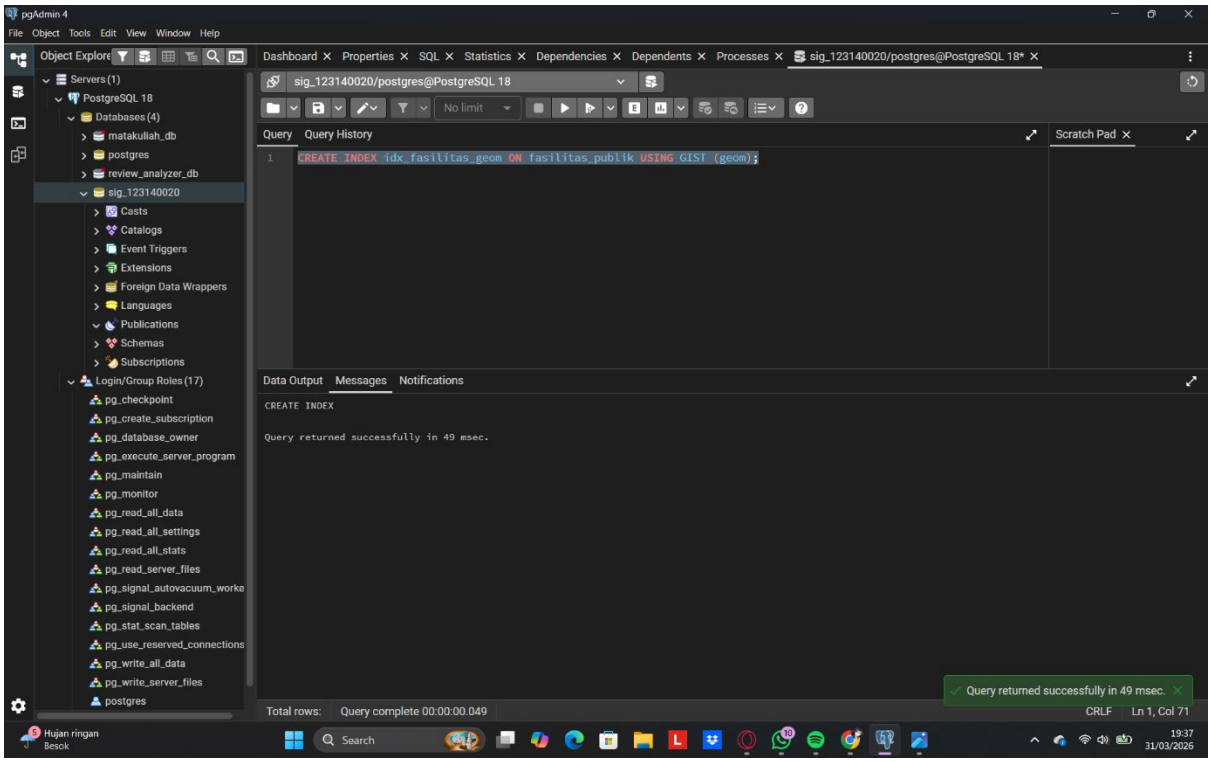
IMPLEMENTASI SPATIAL INDEX (GIST)

Untuk mengubah kompleksitas waktu pencarian dari O(n) menjadi O(log n), struktur indeks berbasis GiST disuntikkan ke dalam kolom geom pada tiga tabel yang berbeda, lalu diikuti dengan perintah pembaruan statistik secara manual.

3.1. Pembuatan Indeks pada Tabel Fasilitas Publik

Query:

```
CREATE INDEX idx_fasilitas_geom ON fasilitas_publik USING GIST (geom);
```



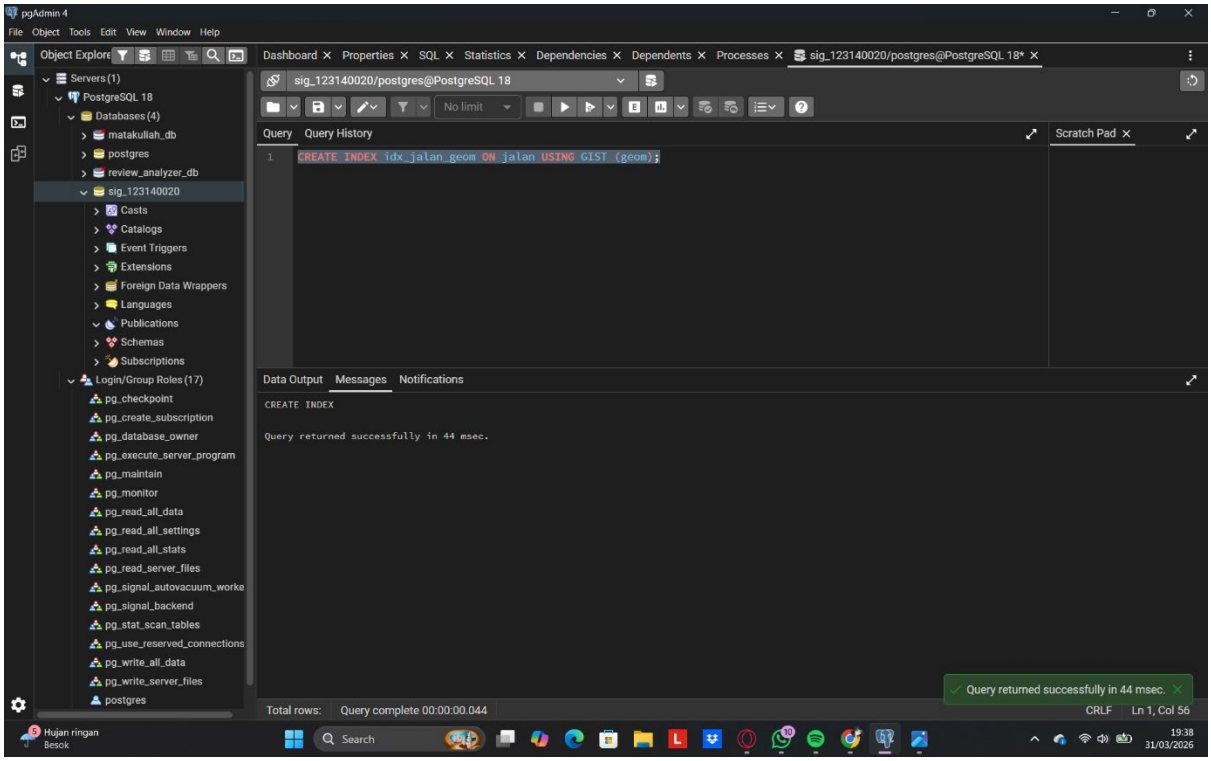
Gambar 3.1. *Implementasi GiST Index pada Tabel Fasilitas Publik*

Implementasi GiST Index pada Tabel Fasilitas Publik **Analisis:** Perintah berhasil dieksekusi, menandakan PostgreSQL telah membuat struktur hierarki *Bounding Box* untuk data koordinat titik (*Point*).

3.2. Pembuatan Indeks pada Tabel Jalan

Query:

```
CREATE INDEX idx_jalan_geom ON jalan USING GIST (geom);
```



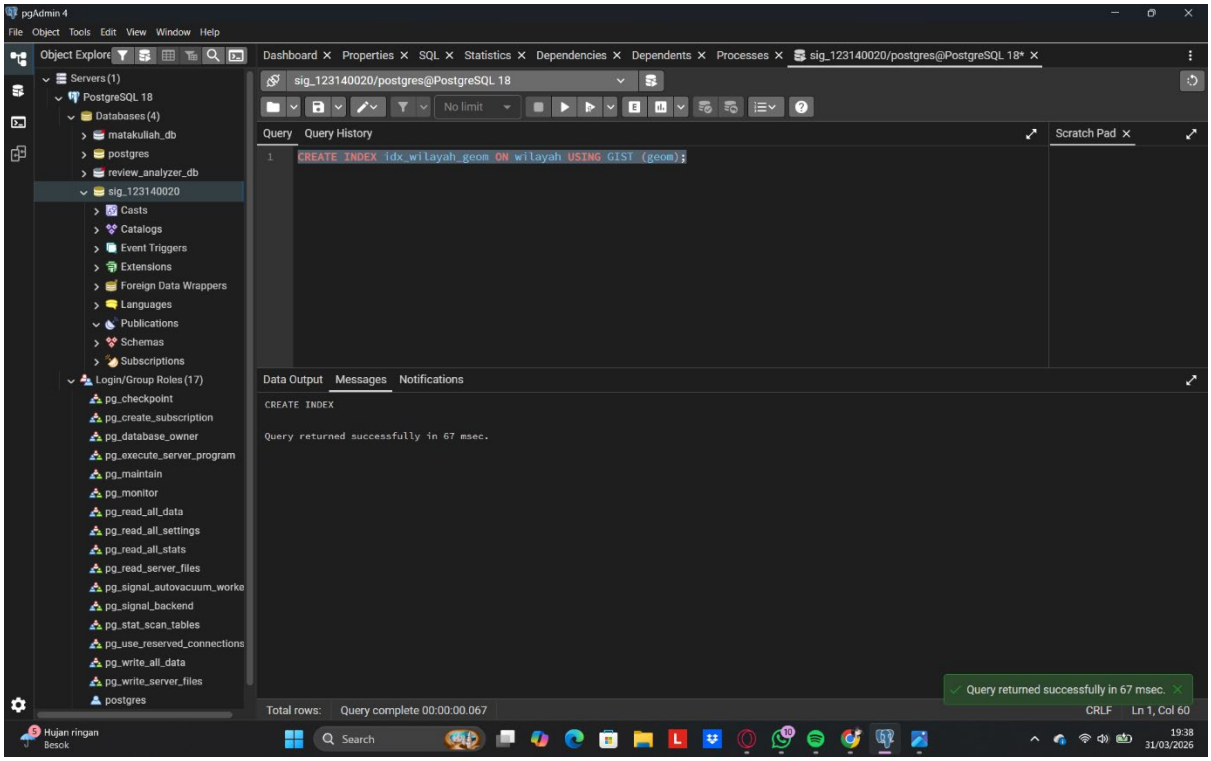
Gambar 3.2. *Implementasi GiST Index pada Tabel Jalan*

Indeks spasial berhasil disematkan pada data berjenis garis (*LineString*), sehingga pencarian rute atau batas jalan tidak perlu memindai keseluruhan tabel.

3.3. Pembuatan Indeks pada Tabel Wilayah

Query:

```
CREATE INDEX idx_wilayah_geom ON wilayah USING GIST (geom);
```



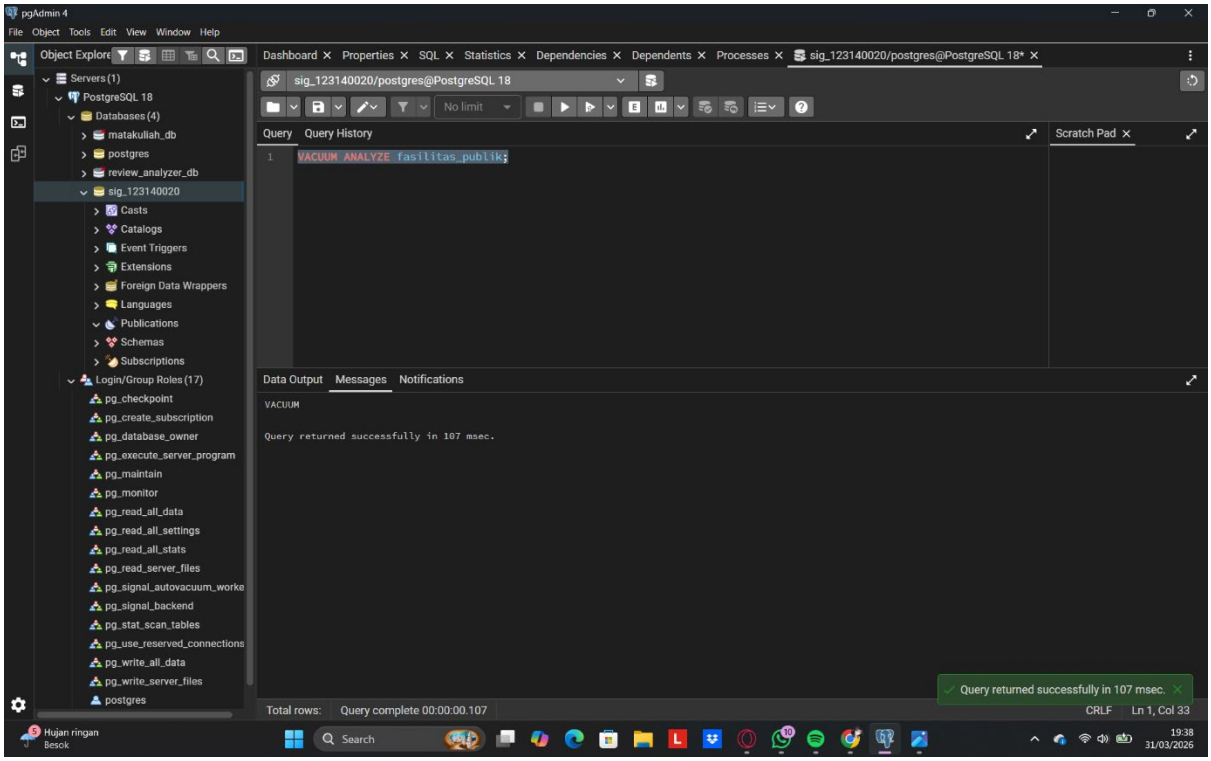
Gambar 3.3. *Implementasi GiST Index pada Tabel Wilayah*

Pembentukan *R-Tree* untuk data poligon berhasil diselesaikan, yang akan sangat membantu mempercepat pengecekan tumpang tindih area luasan ke depannya.

3.4. Pembaruan Statistik Tabel Fasilitas Publik

Query:

VACUUM ANALYZE fasilitas_publik;



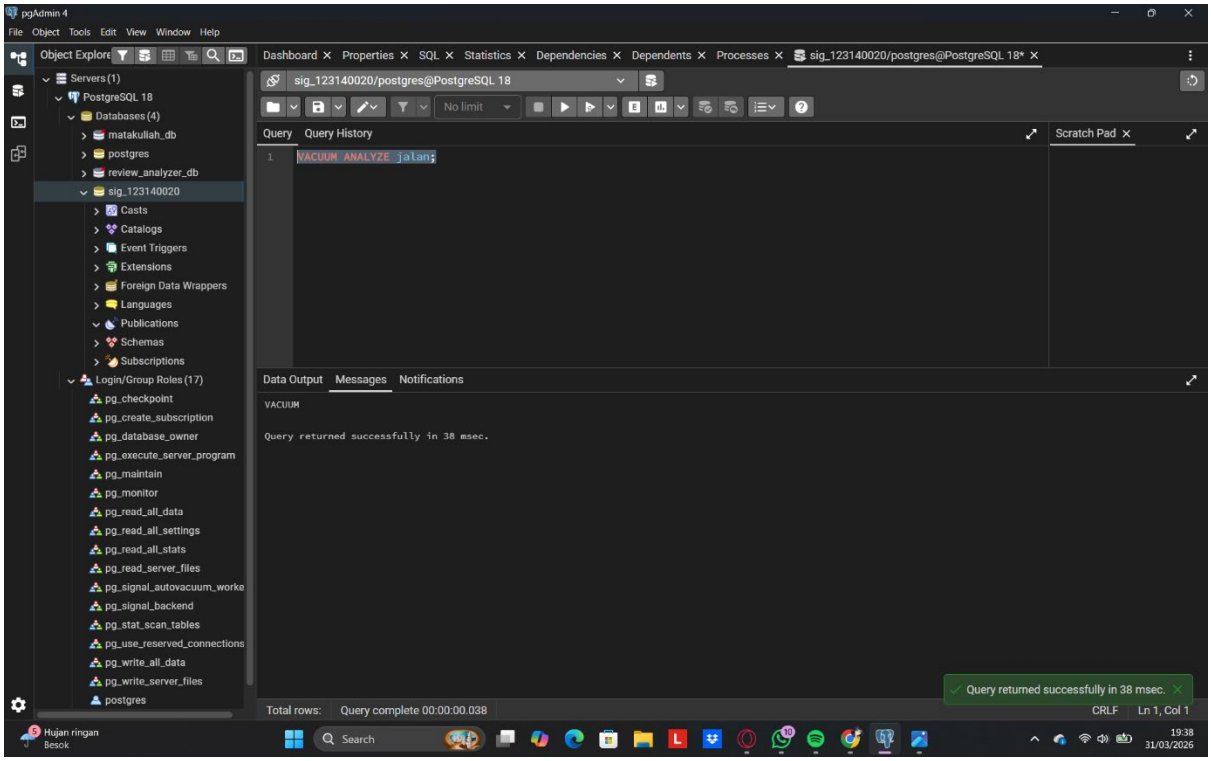
Gambar 3.4. *Eksekusi VACUUM ANALYZE pada Tabel Fasilitas Publik*

Perintah ini wajib dipanggil secara terpisah untuk memaksa mesin PostgreSQL memperbarui peta statistik tabel, sehingga *Query Planner* menyadari adanya indeks baru yang siap digunakan.

3.5. Pembaruan Statistik Tabel Jalan

Query:

VACUUM ANALYZE jalan;



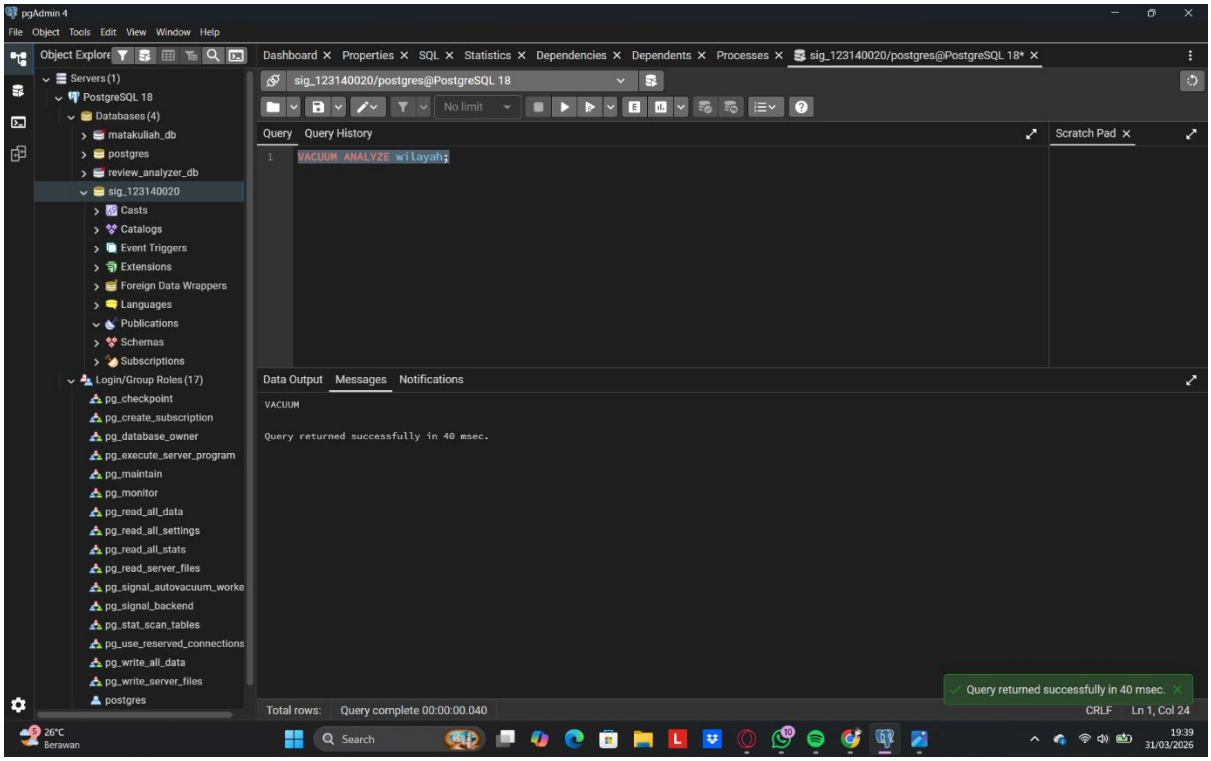
Gambar 3.5. *Eksekusi VACUUM ANALYZE pada Tabel Jalan*

Proses kalibrasi statistik data jalan selesai tanpa kendala. Sistem kini siap beralih ke metode *Index Scan*.

3.6. Pembaruan Statistik Tabel Wilayah

Query:

VACUUM ANALYZE wilayah;



Gambar 3.6. *Eksekusi VACUUM ANALYZE pada Tabel Wilayah*

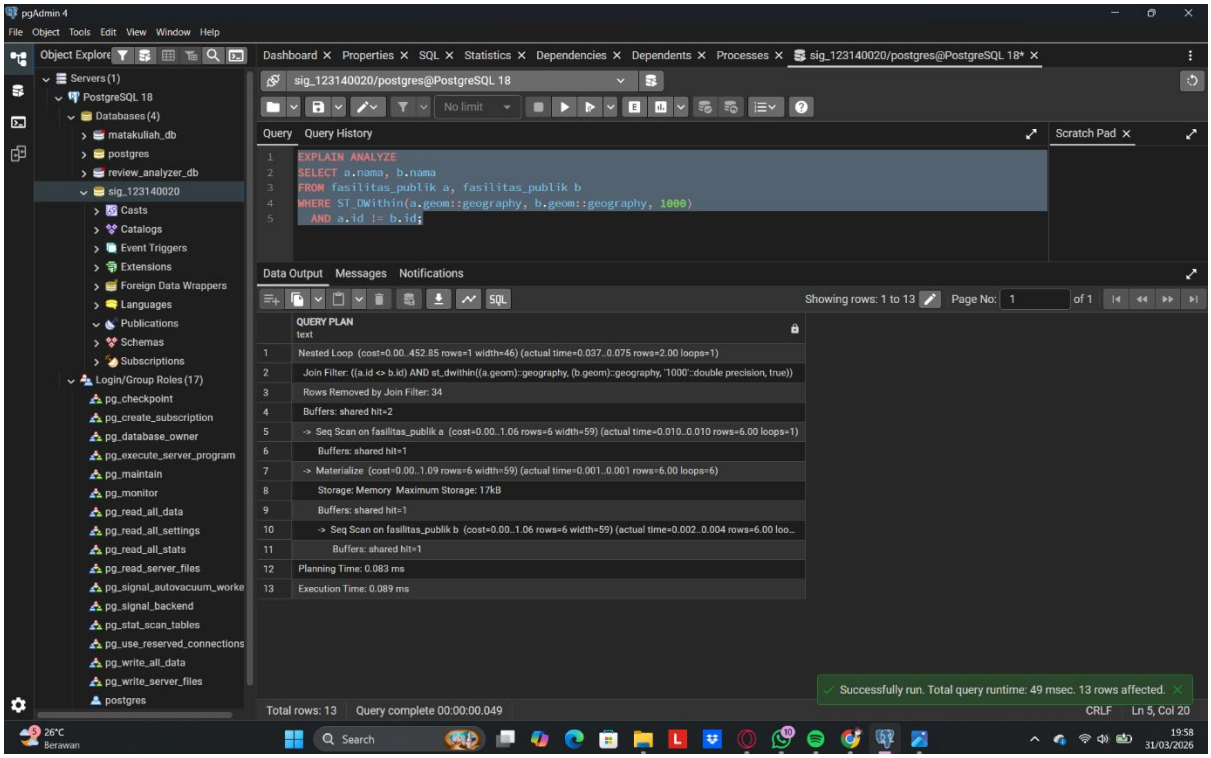
BAB 4.

BENCHMARK AKHIR DAN EVALUASI SPEEDUP

Setelah database terkalibrasi dengan GiST Index, query pencarian yang sama persis seperti pada Bab 2 dieksekusi kembali untuk membuktikan keampuhan metode Index Scan.

4.1. Pasca-Index Fasilitas Publik

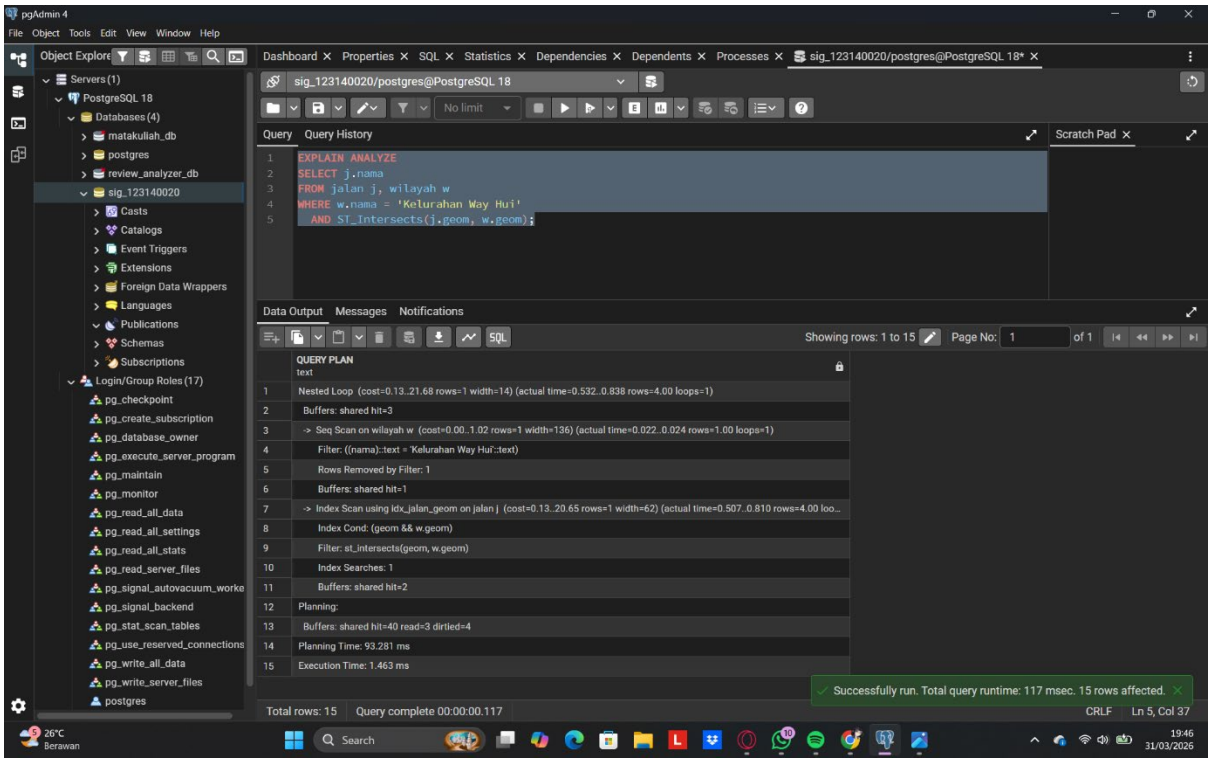
Waktu eksekusi turun drastis dari 0.321 ms menjadi **0.089 ms**.



Gambar 4.1. *Hasil Eksekusi Pasca-Index (Fasilitas Publik)*

4.2. Pasca-Index Jalan vs Wilayah

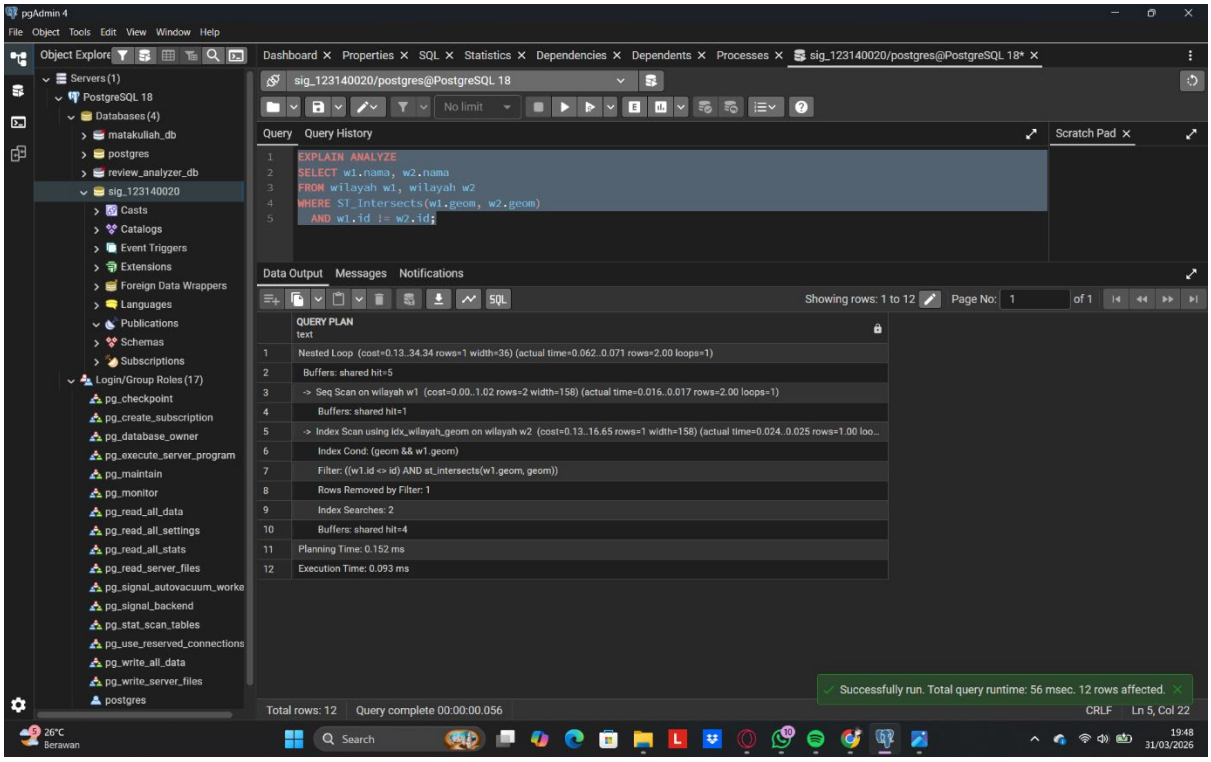
Penurunan yang sangat masif terjadi di sini, dari komputasi berat 28.209 ms anjlok menjadi hanya **1.463 ms**.



Gambar 4.2. *Hasil Eksekusi Pasca-Index (Jalan)*

4.3. Pasca-Index Wilayah vs Wilayah

Waktu eksekusi poligon berkurang secara ekstrem dari 17.098 ms menjadi hanya **0.093 ms**.



Gambar 4.3. *Hasil Eksekusi Pasca-Index (Wilayah)*

4.4. Dokumentasi Peningkatan Performa (Speedup)

Berdasarkan perbandingan waktu eksekusi aktual sebelum dan sesudah optimasi index, didapatkan efisiensi komputasi sebagai berikut:

- Efisiensi Query 1 (Titik): ((0.321 - 0.089) / 0.321) x 100% = 72.27% Lebih Cepat
- Efisiensi Query 2 (Garis vs Poligon): ((28.209 - 1.463) / 28.209) x 100% = 94.81% Lebih Cepat
- Efisiensi Query 3 (Poligon vs Poligon): ((17.098 - 0.093) / 17.098) x 100% = 99.46% Lebih Cepat

Angka-angka ini membuktikan secara empiris bahwa operasi geometri yang kompleks (seperti persinggungan poligon dan garis) memperoleh speedup dan keuntungan terbesar dari keberadaan Spatial Index.

BAB 5.

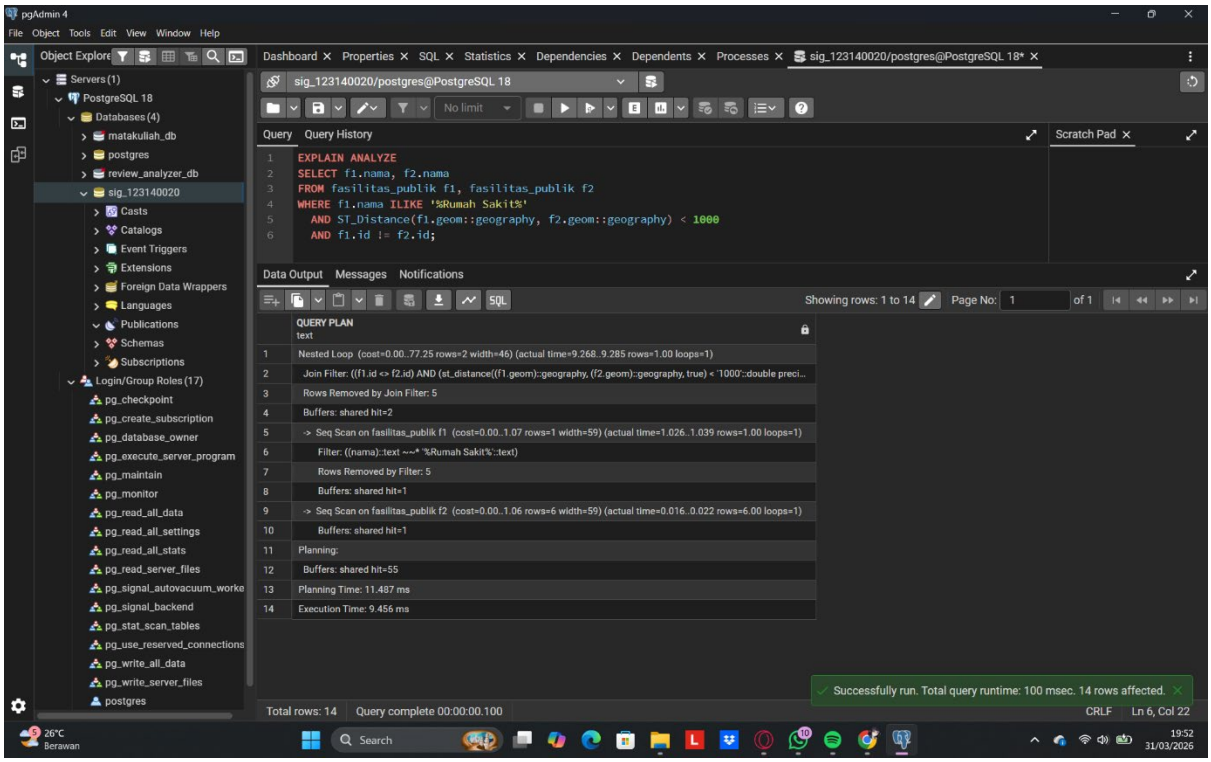
IDENTIFIKASI DAN OPTIMASI QUERY LAMBAT

Meskipun indeks GiST sudah terpasang, penulisan fungsi SQL yang keliru dapat membuat indeks tersebut terabaikan. Kasus yang sering terjadi adalah penggunaan fungsi ST_Distance sebagai *filter* pencarian.

5.1. Identifikasi Query Lambat (Anti-Pattern)

Query:

```
EXPLAIN ANALYZE
SELECT f1.nama, f2.nama FROM fasilitas_publik f1, fasilitas_publik f2
WHERE f1.nama ILIKE '%Rumah Sakit%'
 AND ST_Distance(f1.geom::geography, f2.geom::geography) < 1000
 AND f1.id != f2.id;
```



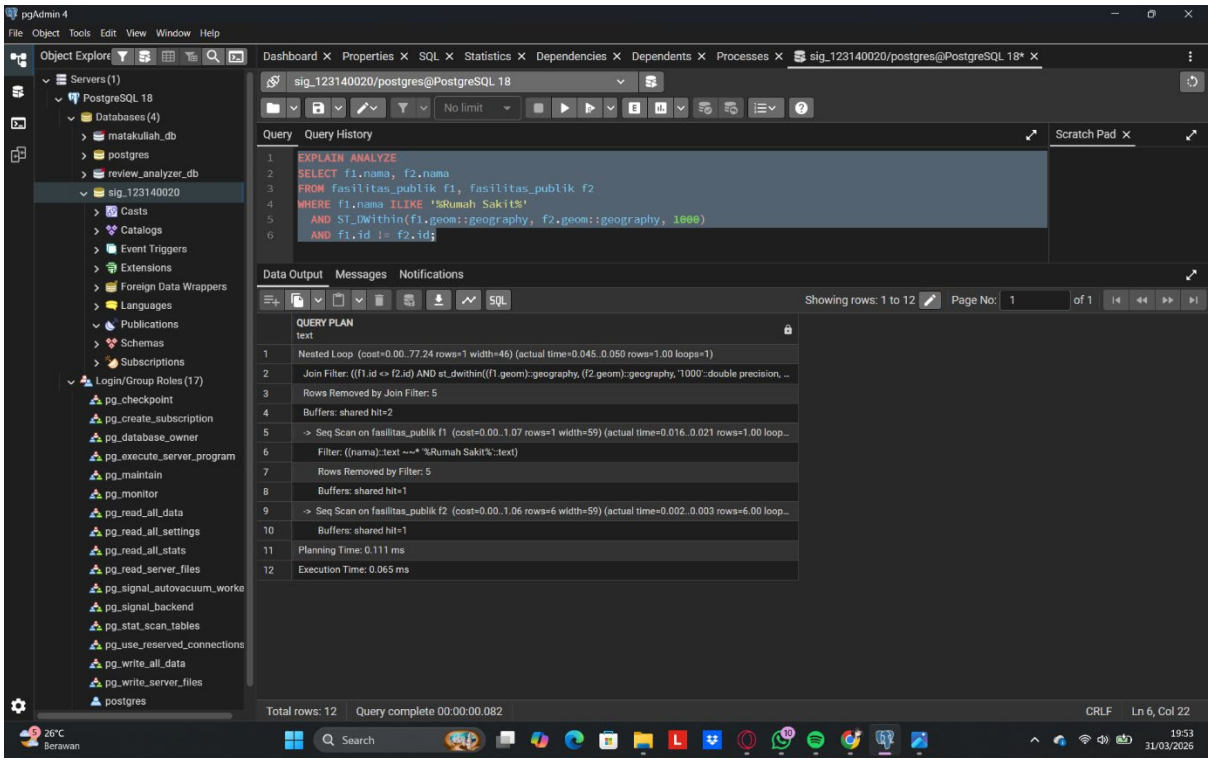
Gambar 5.1. *Pengecekan Jarak menggunakan Anti-Pattern ST_Distance*

Menghasilkan waktu eksekusi yang lambat yaitu **9.456 ms**. Fungsi ST_Distance memaksa sistem menghitung jarak matematis persis antar seluruh titik satu per satu, sehingga keberadaan indeks GiST menjadi percuma (bypassed).

5.2. Optimasi Query (Best Practice)

Query:

```
EXPLAIN ANALYZE
SELECT f1.nama, f2.nama FROM fasilitas_publik f1, fasilitas_publik f2
WHERE f1.nama ILIKE '%Rumah Sakit%'
 AND ST_DWithin(f1.geom::geography, f2.geom::geography, 1000)
 AND f1.id != f2.id;
```



Gambar 5.2. *Pengecekan Jarak menggunakan Best Practice ST_DWithin*

Waktu eksekusi meroket menjadi 0.065 ms. Penggantian fungsi ini memberikan speedup ekstra sebesar 99.3%. Hal ini dimungkinkan karena ST_DWithin dirancang untuk memicu Two-Phase Filter, di mana pencarian menggunakan Bounding Box (&&) untuk menyaring data yang di luar jangkauan dengan sangat cepat sebelum menghitung jarak pastinya.

BAB 6.

KESIMPULAN

Dari keseluruhan proses eksperimen, ditarik kesimpulan bahwa struktur Spatial Index (GiST) adalah komponen arsitektur yang sangat krusial dalam basis data spasial. Data kuantitatif menunjukkan bahwa ketiadaan indeks akan mengakibatkan inefisiensi performa hingga 99%, khususnya saat volume vektor meningkat dan geometri bertambah kompleks (seperti pada poligon dan garis). Namun, keberadaan indeks tersebut juga harus ditunjang dengan pemahaman tata bahasa SQL yang baik, penggunaan fungsi optimasi bounding box seperti ST_DWithin mutlak diperlukan agar query planner dapat benar-benar mengeksploitasi kecepatan R-Tree secara maksimal.

Link Repository GitHub:

https://github.com/forkaton/Praktikum6_SIG_123140020_Anselmus-Herpin-Hasugian.git